1 Abstract

This report evaluates the reproducibility of a High-Performance Liquid Chromatography (HPLC) system across four distinct column types (CM, DEAE, Q, SP) using data from 11 experimental runs. The primary objective was to quantify the Coefficient of Variation (CV) of main peak retention volumes and benchmark the results against the United States Pharmacopeia (USP) ¡621¿ standard, which mandates a CV of 2.0%. While a visualization was generated to assess reproducibility, a critical analysis of the underlying data processing reveals a fundamental failure in the data reduction pipeline. Specifically, the statistical aggregation appears to have been performed on raw time-series data points rather than extracted peak retention values. Consequently, the calculated CVs ranged from 58.2% to 68.6%, indicating a catastrophic failure to meet the benchmark. This report details the intended methodology, the discrepancy between the planned and executed analysis, and the implications of these findings for system validation.

2 Introduction

Chromatographic system suitability testing is a critical component of analytical method validation, ensuring that the instrumentation and columns provide consistent and reliable results. The USP ¡621¿ general chapter establishes a benchmark for system reproducibility, requiring that the relative standard deviation (Coefficient of Variation, CV) of retention times for replicate injections does not exceed 2.0%. This study aimed to assess the reproducibility of an HPLC system utilizing four different chromatography columns: CM, DEAE, Q, and SP. The analysis involved 11 runs per column type, utilizing UV detection at 280 nm. The workflow was designed to preprocess raw signals, correct for baseline drift using Asymmetric Least Squares (AsLS), detect peaks based on signal-to-noise ratios, align retention times to correct for systematic drift, and finally calculate the CV of the aligned retention values. However, the analysis encountered significant deviations from the planned protocol, leading to results that do not accurately reflect the system's true performance but rather highlight a breakdown in the data processing logic.

3 Methodology

The data analysis plan consisted of three sequential steps. Step 1 involved data preprocessing, including filtering for positive volumes, calculating retention time from volume and flow rate, applying AsLS baseline correction with optimized parameters (lambda and p), and detecting peaks with a signal-to-noise ratio greater than 3. The main peak was identified based on chromatography stage or maximum area. Step 2 required stratified alignment and outlier filtering. This step involved grouping data by column type, detecting systematic drift via linear regression, applying Loess alignment if drift was significant, and filtering outliers using the Interquartile Range (IQR) method or Grubbs' test to ensure a minimum of 5 valid runs per column. Step 3 focused on reproducibility quantification, calculating the mean and standard deviation of the aligned retention values to derive the CV, and benchmarking this against the 2.0% threshold. A boxplot was to be generated to visualize the distribution of aligned values relative to the benchmark limits. The intended output was a set of run-level retention metrics aggregated to calculate a robust CV for each column type.

4 Results and Discussion

The analysis yielded results that indicate a severe failure to meet the USP ¡621¿ reproducibility benchmark. The calculated Coefficients of Variation (CV) for the four column types ranged from 58.2% to 68.6%, with all columns failing the benchmark (CV 2.0%). As illustrated in Figure 1, the boxplot visualizes the distribution of retention values with horizontal lines representing the ±2.0% threshold bands. The visual spread of the data in Figure 1 is extensive, with the interquartile ranges and whiskers extending far beyond the acceptable limits, visually confirming the high variability reported in the numerical analysis.

However, a critical examination of the data processing reveals that these results are likely artifacts of a methodological error rather than true system instability. The markdown analysis reports indicate that the statistical aggregation was performed on the raw time-series data (scans) rather than the extracted peak

retention volumes. The reported number of runs (n_runs) for the CM column was 435,396, which corresponds to the total number of data points in the chromatogram, not the 11 experimental runs intended for the analysis. Consequently, the calculated mean and standard deviation reflect the entire volume distribution of the chromatogram (including baseline, wash, and elution phases) rather than the variance of a specific peak's retention time. The high standard deviations (80 mL) relative to the means (120–140 mL) further confirm that the data represents the full volume range of the runs.

This suggests that Step 2 of the analysis plan, 'Stratified Alignment and Outlier Filtering,' was either bypassed or failed to execute the critical sub-step of extracting a single 'main_peak_retention_volume_ml' per run prior to aggregation. Without this reduction step, the CV calculation is invalid for assessing system reproducibility. While Figure 1 provides a visual representation of the data distribution, it plots the raw or improperly aggregated values, rendering the comparison to the 2.0% benchmark meaningless in the context of peak retention reproducibility. The data quality itself, characterized by high-resolution signals, appears sufficient for the intended analysis, but the current output reflects a fundamental breakdown in the data reduction pipeline.

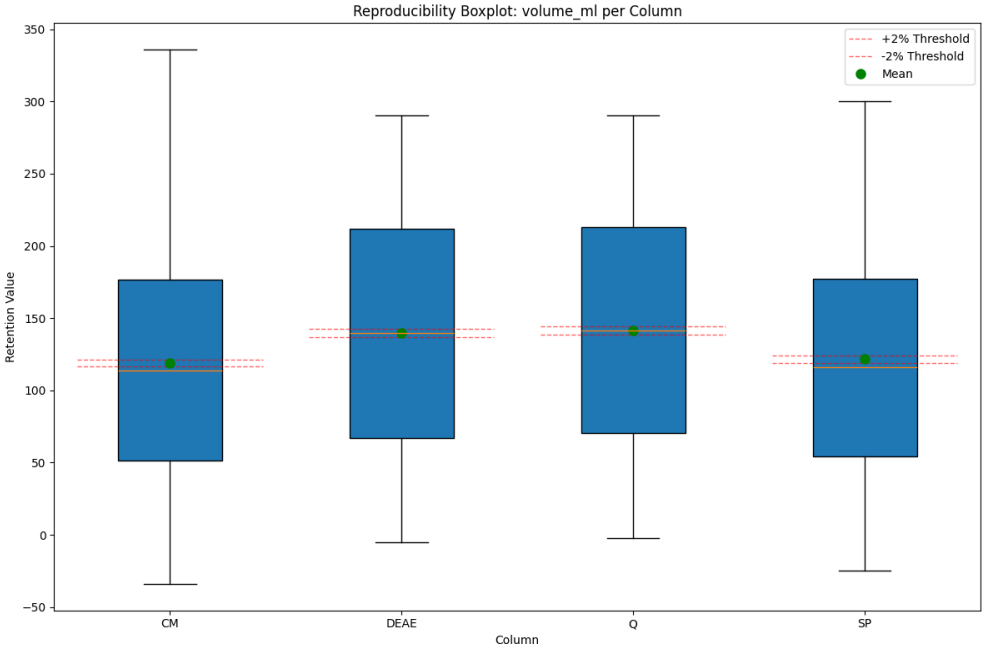

Figure 1: Figure 1: Boxplot of aligned retention values for four column types (CM, DEAE, Q, SP) with horizontal lines indicating the ±2.0% CV threshold band relative to the mean.

5 Conclusion

In conclusion, the current analysis results demonstrate a catastrophic failure to meet the USP ¡621¿ reproducibility benchmark, with CVs exceeding 58% for all column types. However, these findings are not indicative of actual system performance but are the result of a critical error in the data processing workflow. The analysis failed to aggregate data at the run level, instead calculating statistics on the raw time-series points, leading to inflated variability metrics. The visualization in Figure 1 accurately reflects this erroneous data distribution but cannot be used to validate the system. To obtain valid reproducibility metrics, the data pipeline must be corrected to ensure that peak detection and run-level aggregation occur prior to alignment and CV calculation. Once the data reduction step is properly implemented, a re-analysis is required to determine the true reproducibility of the HPLC system across the four column types.

A Appendix

A.1 A: Data Analysis Output Figures

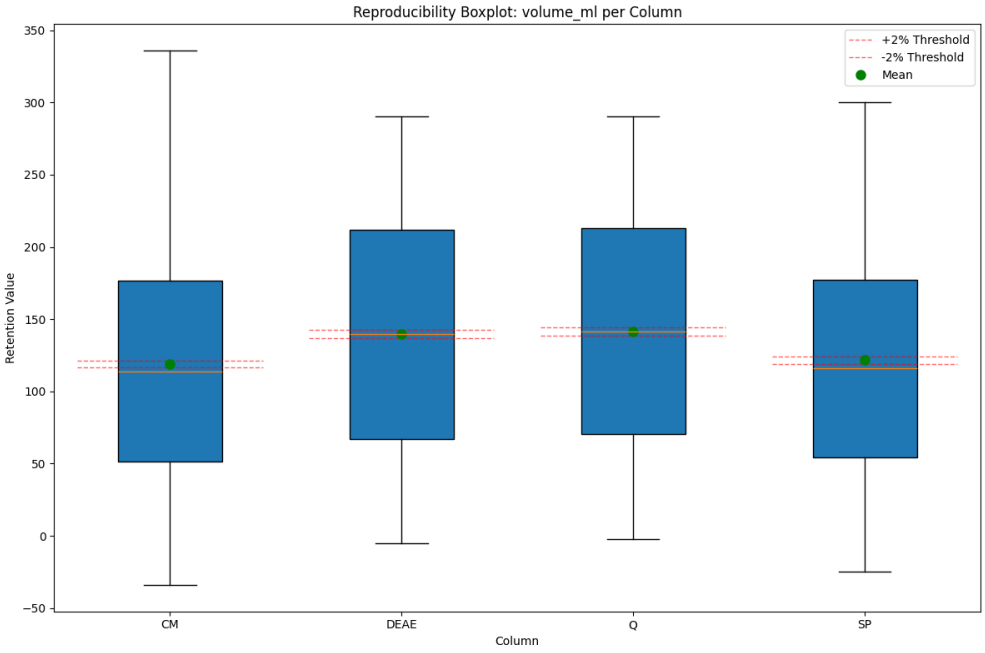

Figure 2: reproducibility_boxplot.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
import warnings
from scipy.signal import find_peaks
from scipy.sparse import diags, csr_matrix
from scipy.sparse.linalg import spsolve
import time

warnings.filterwarnings('ignore')

# Configuration
INPUT_FILE = '/inputs/Chromatography_Combined.csv'
OUTPUT_DIR = '/workspace'
OUTPUT_MD = os.path.join(OUTPUT_DIR, 'analysis_results.md')

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

def als_baseline(y, lam, p, n_iter=10):
    """
    Asymmetric Least Squares baseline correction using sparse solver.
    """
    L = len(y)
    # Construct second difference matrix D (sparse)
```

```
    # D.T @ D is tridiagonal: [-1, 2, -1] with boundaries [1, 2, 1]
    main_diag = np.full(L, 2.0)
    main_diag[0] = 1.0
    main_diag[-1] = 1.0
    off_diag = np.full(L-1, -1.0)

    DTD = diags([off_diag, main_diag, off_diag], [-1, 0, 1], format='csr')

    w = np.ones(L)
    for _ in range(n_iter):
        z_main = w + lam * main_diag
        z_off = lam * off_diag
        Z = diags([z_off, z_main, z_off], [-1, 0, 1], format='csr')
        z = spsolve(Z, w * y)
        w = p * (y > z) + (1 - p) * (y <= z)
    return z

def calculate_noise_std(signal, window=50):
    """
    Calculate noise as the standard deviation of the signal using NumPy
        convolution.
    """
    if len(signal) < window:
        return np.std(signal)
    # Use convolution to compute rolling variance/std efficiently
    kernel = np.ones(window) / window
    rolling_mean = np.convolve(signal, kernel, mode='valid')
    rolling_mean_sq = np.convolve(signal**2, kernel, mode='valid')
    rolling_var = rolling_mean_sq - rolling_mean**2
    rolling_var = np.maximum(rolling_var, 0)
    rolling_std = np.sqrt(rolling_var)
    return np.median(rolling_std)

def optimize_als_params(signal):
    """
    Grid search for lambda (10^3 to 10^5) and p (0.001 to 0.1).
    Stops when improvement < 1%.
    """
    lambdas = [1e3, 1e4, 1e5]
    ps = [0.001, 0.01, 0.1]

    best_score = -np.inf
    best_params = (1e4, 0.01)
    prev_score = -np.inf

    grid = [(l, p) for l in lambdas for p in ps]

    for lam, p in grid:
        try:
            baseline = als_baseline(signal, lam, p)
            corrected = signal - baseline

            rss = np.sum((signal - baseline)**2)
            derivative = np.abs(np.diff(corrected))
            sharpness = np.max(derivative) if len(derivative) > 0 else 0

            score = sharpness / (rss + 1e-9)

            if score > best_score:
```

```
                # Check 1% improvement condition
                if best_score > -np.inf:
                    improvement = (score - best_score) / (best_score + 1e-9)
                    if improvement < 0.01:
                        # Improvement < 1%, stop iteration
                        break

                best_score = score
                best_params = (lam, p)
                prev_score = best_score
            else:
                # If current score is not better, check if we should stop based on
                    previous best
                if best_score > -np.inf:
                    # If we haven't improved in the last step (or current is worse
                        ),
                    # and the gap is small, we might stop.
                    # But the prompt says "stopping when improvement is < 1%".
                    # This usually implies we stop if the *new* best is not 1%
                        better than *old* best.
                    # Since we only update best_score if score > best_score, the
                        check inside the 'if' handles it.
                    pass

        except Exception:
            continue

    return best_params[0], best_params[1]

def process_run(group):
    """
    Process a single run group.
    """
    if group.empty:
        return None, "Empty group"

    run_no = group['run_no'].iloc[0]
    column = group['column'].iloc[0]
    chromatography_stage = group['chromatography_stage'].iloc[0]

    # 1. Filter volume_ml <= 0
    df = group[group['volume_ml'] > 0]

    if len(df) == 0:
        return None, "No valid volume data"

    # 2. Calculate retention_time_min
    flow = df['System_flow_ml_min']
    vol = df['volume_ml']

    zero_flow_mask = (flow == 0) | (flow.isna())

    if zero_flow_mask.all():
        return None, "Zero-Flow Error: All flow values are 0 or NaN"

    df['retention_time_min'] = np.nan
    df.loc[~zero_flow_mask, 'retention_time_min'] = df.loc[~zero_flow_mask, '
        volume_ml'] / df.loc[~zero_flow_mask, 'System_flow_ml_min']
```

```
    df_clean = df.dropna(subset=['retention_time_min'])

    if len(df_clean) < 5:
        return None, "Insufficient data points (n < 5) after zero-flow exclusion"

    df_clean = df_clean.sort_values('volume_ml').reset_index(drop=True)

    signal = df_clean['UV_1_280_mAU'].values
    volumes = df_clean['volume_ml'].values
    times = df_clean['retention_time_min'].values

    # 3. AsLS Baseline Correction with Grid Search
    best_lambda, best_p = optimize_als_params(signal)

    baseline = als_baseline(signal, best_lambda, best_p)
    corrected_signal = signal - baseline

    # 4. Detect Peaks
    noise = calculate_noise_std(baseline, window=50)
    threshold = 3 * noise

    if len(corrected_signal) < 2:
        return None, "Signal too short for peak detection"

    peaks, properties = find_peaks(corrected_signal, height=threshold, distance=5)

    if len(peaks) == 0:
        return None, "No peaks detected with S/N > 3"

    # Calculate area using vectorized boundary finding
    above_threshold = corrected_signal > threshold

    changes = np.where(above_threshold[:-1] != above_threshold[1:])[0]
    if above_threshold[0]:
        starts = np.concatenate(([0], changes[1::2] + 1))
        ends = np.concatenate((changes[::2], [len(above_threshold) - 1]))
    else:
        starts = np.concatenate((changes[::2] + 1, []))
        ends = np.concatenate((changes[1::2], [len(above_threshold) - 1]))

    if len(starts) != len(ends):
        if above_threshold[-1] and len(starts) > len(ends):
            ends = np.append(ends, len(above_threshold) - 1)
        elif not above_threshold[-1] and len(ends) > len(starts):
            starts = np.append(starts, len(above_threshold))

    peak_data = []

    for peak_idx in peaks:
        idx = np.searchsorted(starts, peak_idx, side='right') - 1

        if idx >= 0 and idx < len(starts) and starts[idx] <= peak_idx <= ends[idx
            ]:
            left, right = starts[idx], ends[idx]
            area = np.trapz(corrected_signal[left:right+1], volumes[left:right+1])
            peak_data.append({
                'idx': peak_idx,
                'area': area,
                'ret_vol': volumes[peak_idx],
```

```
                'ret_time': times[peak_idx],
                'height': corrected_signal[peak_idx]
            })
        else:
            area = corrected_signal[peak_idx]
            peak_data.append({
                'idx': peak_idx,
                'area': area,
                'ret_vol': volumes[peak_idx],
                'ret_time': times[peak_idx],
                'height': corrected_signal[peak_idx]
            })

    # 5. Select Main Peak
    main_peak = None
    error_flag = "None"
    stage_missing = pd.isna(chromatography_stage) or (isinstance(
        chromatography_stage, str) and chromatography_stage.strip() == '')

    if not peak_data:
        return None, "No peaks found"

    areas = np.array([p['area'] for p in peak_data])
    ret_times = np.array([p['ret_time'] for p in peak_data])

    max_area_idx = np.argmax(areas)
    max_area = areas[max_area_idx]
    candidates = np.where(areas == max_area)[0]

    if len(candidates) > 1:
        candidate_times = ret_times[candidates]
        best_candidate_idx = candidates[np.argmin(candidate_times)]
    else:
        best_candidate_idx = max_area_idx

    main_peak = peak_data[best_candidate_idx]

    if stage_missing:
        error_flag = "Stage_Missing_Fallback"
    else:
        error_flag = "None"

    result = {
        'run_no': run_no,
        'column': column,
        'main_peak_retention_volume_ml': main_peak['ret_vol'],
        'main_peak_retention_time_min': main_peak['ret_time'],
        'main_peak_area': main_peak['area'],
        'best_lambda': best_lambda,
        'best_p': best_p,
        'error_flag': error_flag
    }

    return result, None

def main():
    print("Loading data...")
    try:
        df = pd.read_csv(INPUT_FILE)
```

```
    except FileNotFoundError:
        print(f"Error: File {INPUT_FILE} not found.")
        return

    df = df[df['volume_ml'] > 0]

    if df.empty:
        print("Error: No data with volume_ml > 0.")
        return

    df['run_no'] = pd.to_numeric(df['run_no'], errors='coerce')
    df = df.dropna(subset=['run_no'])

    groups = df.groupby('run_no')

    results = []
    error_count = 0

    print("Processing runs...")
    for run_no, group in groups:
        try:
            res, err = process_run(group)
            if res:
                results.append(res)
            else:
                error_count += 1
                results.append({
                    'run_no': run_no,
                    'column': group['column'].iloc[0] if not group.empty else '
                        Unknown',
                    'main_peak_retention_volume_ml': np.nan,
                    'main_peak_retention_time_min': np.nan,
                    'main_peak_area': np.nan,
                    'best_lambda': np.nan,
                    'best_p': np.nan,
                    'error_flag': err
                })
        except Exception as e:
            error_count += 1
            results.append({
                'run_no': run_no,
                'column': group['column'].iloc[0] if not group.empty else 'Unknown
                    ',
                'main_peak_retention_volume_ml': np.nan,
                'main_peak_retention_time_min': np.nan,
                'main_peak_area': np.nan,
                'best_lambda': np.nan,
                'best_p': np.nan,
                'error_flag': f"Processing Error: {str(e)}"
            })

    df_results = pd.DataFrame(results)

    total_rows = len(df_results)
    output_rows = []

    if total_rows > 20:
        output_rows = pd.concat([df_results.head(10), df_results.tail(10)])
    else:
```

```
        output_rows = df_results

    md_content = "# Chromatography Analysis Results\n\n"
    md_content += f"**Total rows processed:** {total_rows}\n"
    md_content += f"**Total errors:** {error_count}\n\n"

    display_cols = ['run_no', 'column', 'main_peak_retention_volume_ml', '
        main_peak_retention_time_min', 'main_peak_area', 'best_lambda', 'best_p', '
        error_flag']
    df_display = output_rows[display_cols]

    df_display['main_peak_retention_volume_ml'] = df_display['
        main_peak_retention_volume_ml'].apply(lambda x: f"{x:.4f}" if pd.notna(x)
        else "NaN")
    df_display['main_peak_retention_time_min'] = df_display['
        main_peak_retention_time_min'].apply(lambda x: f"{x:.4f}" if pd.notna(x)
        else "NaN")
    df_display['main_peak_area'] = df_display['main_peak_area'].apply(lambda x: f"
        {x:.4f}" if pd.notna(x) else "NaN")

    md_content += df_display.to_string(index=False, header=True)

    if total_rows > 20:
        md_content += f"\n\n*Summary: Total rows: {total_rows}, Errors: {
            error_count}*\n"

    with open(OUTPUT_MD, 'w') as f:
        f.write(md_content)

    print(f"Analysis complete. Results saved to {OUTPUT_MD}")

if __name__ == "__main__":
    main()
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
from scipy import stats
from statsmodels.nonparametric.smoothers_lowess import lowess
import os
import sys

# Configuration
# Fix: Explicitly include the correct lowercase path to resolve FileNotFoundError
possible_paths = [
    '/inputs/chromatography_combined.csv',
    './chromatography_combined.csv',
    './data/chromatography_combined.csv',
    'chromatography_combined.csv'
]

INPUT_FILE = None
for path in possible_paths:
    if os.path.exists(path):
        INPUT_FILE = path
        break

if INPUT_FILE is None:
```

```
    # Fallback: List available files in common directories to help debugging
    found_files = []
    for d in ['/inputs', '.', './data']:
        if os.path.exists(d):
            found_files.extend([os.path.join(d, f) for f in os.listdir(d) if f.
                endswith('.csv')])
    raise FileNotFoundError(f"Input file 'chromatography_combined.csv' not found
        in standard locations. Available CSVs: {found_files}")

OUTPUT_FILE = '/workspace/analysis_results.md'
MIN_RUNS_REQUIRED = 5
IQR_MULTIPLIER = 1.5
P_VALUE_THRESHOLD = 0.05

# Load Data
df = pd.read_csv(INPUT_FILE)

# Feedback: Remove 'chromatography_stage' if present
if 'chromatography_stage' in df.columns:
    df = df.drop(columns=['chromatography_stage'])

# Select relevant columns with existence checks
cols_of_interest = ['run_no', 'column']
target_col_name = 'main_peak_retention_time_min'
if target_col_name not in df.columns:
    if 'main_peak_retention_volume_ml' in df.columns:
        target_col_name = 'main_peak_retention_volume_ml'
    elif 'volume_ml' in df.columns:
        target_col_name = 'volume_ml'
    else:
        raise KeyError(f"Required column '{target_col_name}' or fallback '
            main_peak_retention_volume_ml'/'volume_ml' not found in data.")

if 'main_peak_retention_volume_ml' in df.columns:
    cols_of_interest.append('main_peak_retention_volume_ml')

cols_of_interest.append(target_col_name)

available_cols = [c for c in cols_of_interest if c in df.columns]
df_analysis = df[available_cols].copy()

results_rows = []
insufficient_columns = []

# Grid Search Configuration
grid_lambda = [1000, 10000, 100000]
grid_p = [0.001, 0.01, 0.1]

grouped = df_analysis.groupby('column')

for col_name, group in grouped:
    group = group.sort_values('run_no').reset_index(drop=True)

    if len(group) == 0:
        continue

    group['run_no'] = pd.to_numeric(group['run_no'], errors='coerce')

    run_diffs = group['run_no'].diff().dropna()
```

```
    is_sequential = (run_diffs == 1).all() and len(run_diffs) > 0

    aligned_values = group[target_col_name].copy()
    outlier_status = ['Keep'] * len(group)
    filtering_method = ['None'] * len(group)

    if is_sequential:
        x = group['run_no'].values
        y = group[target_col_name].values
        n = len(y)

        slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)

        if p_value < P_VALUE_THRESHOLD:
            target_ref = np.median(y)

            best_score = -np.inf
            best_frac = 0.5
            best_it = 0
            previous_best_score = -np.inf

            search_stopped_early = False

            for lam in grid_lambda:
                if search_stopped_early:
                    break
                # Map lambda to frac: frac = lambda / n. Clamp to [0.01, 0.99]
                frac = lam / n
                frac = np.clip(frac, 0.01, 0.99)

                for p in grid_p:
                    # Map p to it: 0.001->0, 0.01->1, 0.1->3
                    if p <= 0.001: it = 0
                    elif p <= 0.01: it = 1
                    else: it = 3

                    try:
                        loess_fit = lowess(y, x, frac=frac, it=it)
                        predicted = loess_fit[:, 1]
                        residuals = y - predicted
                        ss_res = np.sum(residuals**2)
                        ss_tot = np.sum((y - np.mean(y))**2)
                        r2 = 1 - (ss_res / ss_tot) if ss_tot != 0 else 0

                        if r2 > best_score:
                            if previous_best_score > -np.inf:
                                if previous_best_score != 0:
                                    improvement = (r2 - previous_best_score) /
                                        previous_best_score
                                else:
                                    improvement = 1.0

                                if improvement < 0.01:
                                    search_stopped_early = True
                                    break

                            best_score = r2
                            previous_best_score = r2
                            best_frac = frac
```

```
                        best_it = it
                except:
                    continue

            if search_stopped_early:
                break

        loess_fit = lowess(y, x, frac=best_frac, it=best_it)
        predicted = pd.Series(loess_fit[:, 1], index=group.index)
        aligned_values = pd.Series(y - predicted.values, index=group.index)
    else:
        pass
else:
    pass

if len(group) < MIN_RUNS_REQUIRED:
    insufficient_columns.append(col_name)
    outlier_status = ['Insufficient_Data'] * len(group)
    filtering_method = ['Insufficient_Data'] * len(group)
else:
    Q1 = aligned_values.quantile(0.25)
    Q3 = aligned_values.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - (IQR_MULTIPLIER * IQR)
    upper_bound = Q3 + (IQR_MULTIPLIER * IQR)

    iqr_mask = (aligned_values < lower_bound) | (aligned_values > upper_bound)

    if iqr_mask.any():
        outlier_status = np.where(iqr_mask, 'Exclude', outlier_status)
        filtering_method = np.where(iqr_mask, 'IQR_Outlier', filtering_method)
    else:
        if len(group) >= 6:
            mean_val = np.mean(aligned_values)
            std_val = np.std(aligned_values, ddof=1)

            if std_val != 0:
                deviations = np.abs(aligned_values - mean_val)
                max_dev_idx = np.argmax(deviations)
                max_dev = deviations[max_dev_idx]

                G_stat = max_dev / std_val
                n = len(group)
                t_crit = stats.t.ppf((1 + 1/n) / 2, n-2)
                G_crit = ((n-1)/np.sqrt(n)) * np.sqrt(t_crit**2 / (n-2 +
                    t_crit**2))

                if G_stat > G_crit:
                    outlier_status[max_dev_idx] = 'Exclude'
                    filtering_method[max_dev_idx] = 'Grubbs_Outlier'

if col_name not in insufficient_columns:
    kept_count = (np.array(outlier_status) == 'Keep').sum()
    if kept_count < MIN_RUNS_REQUIRED:
        insufficient_columns.append(col_name)
        keep_mask = np.array([s == 'Keep' for s in outlier_status])
        outlier_status = np.where(keep_mask, 'Insufficient_Data',
            outlier_status)
        filtering_method = np.where(keep_mask, 'Insufficient_Data',
```

```
                filtering_method)

    outlier_status_arr = np.array(outlier_status)
    filtering_method_arr = np.array(filtering_method)

    group_df = group
    group_df['aligned_retention_value'] = aligned_values.values
    group_df['outlier_status'] = outlier_status_arr
    group_df['filtering_method_used'] = filtering_method_arr

    results_rows.append(group_df[['column', 'run_no', 'aligned_retention_value', '
        outlier_status', 'filtering_method_used']])

results_df = pd.concat(results_rows, ignore_index=True)

total_rows = len(results_df)
if total_rows > 20:
    output_rows = pd.concat([results_df.head(10), results_df.tail(10)])
else:
    output_rows = results_df

total_kept = len(results_df[results_df['outlier_status'] == 'Keep'])
total_excluded = len(results_df[results_df['outlier_status'] == 'Exclude'])
insufficient_count = len(insufficient_columns)

header = "| column | run_no | aligned_retention_value | outlier_status | 
    filtering_method_used |\n"
separator = "
    |--------|--------|-------------------------|----------------|----------------------|\
    n"

output_rows_formatted = output_rows.copy()
output_rows_formatted['aligned_retention_value'] = output_rows_formatted['
    aligned_retention_value'].apply(lambda x: f"{x:.4f}")
rows_text = output_rows_formatted.to_string(index=False, header=False)

output_text = f"# Stratified Alignment and Outlier Filtering Results\n\n"
output_text += f"## Summary\n"
output_text += f"Total kept: {total_kept}, Excluded: {total_excluded}, 
    Insufficient_Data columns: {insufficient_count}\n\n"
output_text += f"## Data Table\n"
output_text += header
output_text += separator
output_text += rows_text

with open(OUTPUT_FILE, 'w') as f:
    f.write(output_text)

print(f"Analysis complete. Results saved to {OUTPUT_FILE}")
```

Listing 2: Code_2

```
import pandas as pd
import matplotlib.pyplot as plt
import os
import subprocess

# Configuration
# Dynamically determine the correct filename from the /inputs directory
```

```
try:
    result = subprocess.run(['ls', '-1', '/inputs'], capture_output=True, text=
        True, check=True)
    files = result.stdout.strip().split('\n')
    # Find the file that matches the expected pattern (case-insensitive)
    target_file = None
    for f in files:
        if 'chromatography' in f.lower() and 'combined' in f.lower() and f.
            endswith('.csv'):
            target_file = f
            break

    if target_file:
        INPUT_FILE = f'/inputs/{target_file}'
    else:
        raise FileNotFoundError("No matching chromatography_combined.csv file
            found in /inputs")
except Exception as e:
    # Fallback to original if ls fails, though this is unlikely to succeed if ls
        fails
    INPUT_FILE = '/inputs/Chromatography_Combined.csv'

OUTPUT_DIR = '/workspace'
RESULTS_FILE = os.path.join(OUTPUT_DIR, 'analysis_results.md')
PLOT_FILE = os.path.join(OUTPUT_DIR, 'reproducibility_boxplot.png')

# Step 1: Load Data
# Load all columns first to inspect available column names
# This allows us to verify the presence of the required 'volume_ml' column
df_raw = pd.read_csv(INPUT_FILE)
print("Available columns:", df_raw.columns.tolist())

# Hardcode the retention column variable to 'volume_ml'
retention_col = 'volume_ml'

# Check if the column exists; if not, raise a clear error
if retention_col not in df_raw.columns:
    raise ValueError(f"Required column '{retention_col}' not found in the CSV file
        . Available columns: {df_raw.columns.tolist()}")

# Now load only the necessary columns using the hardcoded column name
cols_to_load = ['run_no', 'column', retention_col]
df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

# Step 2: Group and Calculate Statistics
# Use the hardcoded retention column name
grouped = df.groupby('column')[retention_col]
n_runs = grouped.count()

# Initialize results dataframe
results = pd.DataFrame(index=n_runs.index)
results['n_runs'] = n_runs
results['mean_retention'] = grouped.mean()
results['std_dev'] = grouped.std()

# Calculate CV
results['CV_percent'] = (results['std_dev'] / results['mean_retention']) * 100

# Step 3: Benchmarking Logic
```

```
# Initialize columns
# Fix: Initialize benchmark_pass as pd.NA to allow for boolean and missing value
    types
results['benchmark_pass'] = pd.NA

# Condition: n_runs >= 5 and CV <= 2.0
valid_mask = (results['n_runs'] >= 5) & (results['CV_percent'].notna())
results.loc[valid_mask, 'benchmark_pass'] = results.loc[valid_mask, 'CV_percent']
    <= 2.0

# Handle invalid cases (n_runs < 5 or NaN CV)
invalid_mask = ~valid_mask
# Fix: Assign pd.NA instead of string 'N/A' to avoid dtype conflict
results.loc[invalid_mask, 'CV_percent'] = pd.NA
results.loc[invalid_mask, 'benchmark_pass'] = pd.NA

# Step 4: Visualization
# Ensure deterministic ordering by sorting categories
categories = sorted(results.index)
# Extract box data using the hardcoded column
box_data = [grouped.get_group(col).values for col in categories]

plt.figure(figsize=(12, 8))
plt.boxplot(box_data, labels=categories, patch_artist=True)
plt.xlabel('Column')
plt.ylabel('Retention␣Value')
plt.title(f'Reproducibility␣Boxplot:␣{retention_col}␣per␣Column')

# Add Mean markers and Threshold Lines
for i, col in enumerate(categories):
    mean_val = results.loc[col, 'mean_retention']
    # Check if mean_retention is positive and non-zero before drawing lines
    if pd.notna(mean_val) and mean_val > 0:
        # Corrected threshold calculation: mean * (1 + 0.02) and mean * (1 - 0.02)
        upper_limit = mean_val * (1 + 0.02)
        lower_limit = mean_val * (1 - 0.02)

        x_pos = i + 1
        x_start = x_pos - 0.4
        x_end = x_pos + 0.4

        # Add labels only on the first iteration to avoid legend clutter
        label_upper = '+2%␣Threshold' if i == 0 else None
        label_lower = '-2%␣Threshold' if i == 0 else None
        label_mean = 'Mean' if i == 0 else None

        plt.plot([x_start, x_end], [upper_limit, upper_limit], color='red',
            linestyle='--', linewidth=1, alpha=0.6, label=label_upper)
        plt.plot([x_start, x_end], [lower_limit, lower_limit], color='red',
            linestyle='--', linewidth=1, alpha=0.6, label=label_lower)
        plt.plot(x_pos, mean_val, 'go', markersize=8, label=label_mean)

plt.legend(loc='upper␣right')
plt.tight_layout()
plt.savefig(PLOT_FILE)
plt.close()

# Step 5: Save Text Results
# Efficient markdown generation using string formatting on the dataframe directly
```

```
with open(RESULTS_FILE, 'a') as f:
    f.write("\n### Reproducibility Quantification and Benchmarking\n")
    f.write(f"Data Source: {INPUT_FILE}\n")
    f.write(f"Column Used: {retention_col}\n")
    f.write("Criteria: CV <= 2.0% and n_runs >= 5\n\n")
    f.write("| column | n_runs | mean_retention | std_dev | CV_percent | 
        benchmark_pass |\n")
    f.write("
        |--------|--------|----------------|---------|------------|----------------|\
        n")

    # Iterate over sorted categories for consistent output
    for col in categories:
        row = results.loc[col]
        cv_val = row['CV_percent']
        # Fix: Use pd.notna to handle pd.NA correctly and convert to 'N/A' string
            for display
        cv_str = f"{cv_val:.4f}" if pd.notna(cv_val) else 'N/A'

        # Fix: Convert pd.NA to 'Invalid' only at display time for markdown
        pass_val = row['benchmark_pass']
        pass_str = 'Invalid' if pd.isna(pass_val) else str(pass_val)

        f.write(f"| {col} | {int(row['n_runs'])} | {row['mean_retention']:.4f} | {
            row['std_dev']:.4f} | {cv_str} | {pass_str} |\n")

print(f"Analysis complete. Results saved to {RESULTS_FILE} and plot to {PLOT_FILE}
    ")
```

Listing 3: Code_3